

lec12

April 24, 2026

1 Programming Languages and Environments

1.1 (Lecture 12)

1.1.1 Agenda

- N-ary trees

1.2 N-ary trees

In an n-ary tree (`ntree`) each node can have **any number of children**, represented as a list of subtrees. Consider the inductive type defined below and the example `nt` — a tree with root 1, subtrees with roots 2, 4 and 5, where the subtree with value 2 at the root has a subtree with value 3.

```
[6]: type 'a ntree = NEmpty | NNode of 'a * 'a ntree list  
  
let nt = NNode (1, [NNode (2, [NNode (3, [])]); NNode (4, []); NNode (5, [])])
```

```
[6]: type 'a ntree = NEmpty | NNode of 'a * 'a ntree list
```

```
[6]: val nt : int ntree =  
      NNode (1, [NNode (2, [NNode (3, [])]); NNode (4, []); NNode (5, [])])
```

This alternative type definition is more precise in the sense that there is no ambiguity between using an empty tree in the subtrees or an empty list. It always requires that all functions have cases to handle the empty tree at the top level.

```
[ ]: type 'a netree = NNode of 'a * 'a netree list  
  
type 'a nntree = NEmpty | NNEmpty of 'a netree
```

```
[ ]: type 'a netree = NNode of 'a * 'a netree list
```

```
[ ]: type 'a nntree = NEmpty | NNEmpty of 'a netree
```

1.2.1 Sum of all values in an n-ary tree — version 1 (with explicit auxiliary function)

`sum_tree` sums all values in the tree. The auxiliary function `sum_aux` iterates over the list of children, calling `sum_tree` on each subtree.

```
[9]: let rec sum_tree t =  
      let rec sum_aux l =  
        match l with  
        | [] -> 0  
        | t'::ts -> sum_tree t' + sum_aux ts  
      in  
      match t with  
      | NEmpty -> 0  
      | NNode(v, l) -> v + sum_aux l  
  
      let _ = assert (sum_tree nt = 15)
```

```
[9]: val sum_tree : int ntree -> int = <fun>
```

```
[9]: - : unit = ()
```

1.2.2 Sum of values — version 2 (with `fold_left`)

A more compact version that replaces the explicit auxiliary function with `List.fold_left`. Note: the initial accumulator is `v` (the value of the current node), which can be confusing — compare with version 4.

```
[ ]: let rec sum_tree t =  
      match t with  
      | NEmpty -> 0  
      | NNode(v, l) -> List.fold_left (fun acc t' -> acc + sum_tree t') v l  
  
      let _ = assert (sum_tree nt = 15)
```

```
[ ]: val sum_tree : int ntree -> int = <fun>
```

```
[ ]: - : unit = ()
```

```
[ ]: val sum_tree : int ntree -> int = <fun>
```

```
[ ]: - : unit = ()
```

We can also use a simpler form, first mapping over the list of subtrees and then summing the collected values.

```
[12]: let rec sum_tree t =
  match t with
  | NEmpty -> 0
  | NNode(v, l) -> List.map sum_tree l |> List.fold_left (+) v

let _ = assert (sum_tree nt = 15)
```

```
[12]: val sum_tree : int ntree -> int = <fun>
```

```
[12]: - : unit = ()
```

1.2.3 Mapping over n-ary trees — version 1

`map_ntree f t` applies `f` to each value in the tree preserving the structure. For each node, it applies `f` to the value and recursively maps over the list of children using an explicit lambda (`fun t' -> map_ntree f t'`).

```
[14]: let rec map_ntree f nt =
  match nt with
  | NEmpty -> NEmpty
  | NNode (v, l) -> NNode (f v, List.map (fun t' -> map_ntree f t') l)
```

```
[14]: val map_ntree : ('a -> 'b) -> 'a ntree -> 'b ntree = <fun>
```

```
[18]: let _ = assert (map_ntree ((+)1) nt = NNode (2, [NNode (3, [NNode (4, [])]);
↪NNode (5, []); NNode (6, [])]))
```

```
[18]: - : unit = ()
```

1.2.4 Mapping — version 2 (point-free style)

A more concise version where the anonymous function `fun t' -> map_ntree f t'` becomes `map_ntree f`, which denotes the function waiting for the argument representing the tree.

```
[42]: let rec map_ntree f nt =
  match nt with
  | NEmpty -> NEmpty
  | NNode (v, l) -> NNode (f v, List.map (map_ntree f) l)
```

```
[42]: val map_ntree : ('a -> 'b) -> 'a ntree -> 'b ntree = <fun>
```

```
[19]: let _ = assert (map_ntree ((+)1) nt = NNode (2, [NNode (3, [NNode (4, [])]);
↪NNode (5, []); NNode (6, [])]))
```

```
[19]: - : unit = ()
```

1.2.5 Pre-order fold over n-ary trees — version 1 (recursive auxiliary function)

`prefix_fold_ntree f acc t` traverses the tree using **pre-order** (root before subtrees). Variant with the auxiliary function `fold_aux` that manually iterates over the list of children, without using `List.fold_left`. Makes the accumulation pattern more visible.

```
[34]: let rec prefix_fold_ntree f acc nt =  
      let rec fold_aux acc l =  
        match l with  
        | [] -> acc  
        | nt'::nts -> let acc_nt' = prefix_fold_ntree f acc nt' in fold_aux acc_nt'   
      ↪ nts  
      in  
      match nt with  
      | NEmpty -> acc  
      | NNode (v, l) -> let acc_v = f acc v in fold_aux acc_v l  
  
      let _ = assert (prefix_fold_ntree (+) 0 nt = 15)  
      let _ = assert (prefix_fold_ntree (fun acc x -> acc@[x]) [] nt = [1; 2; 3; 4;   
      ↪ 5])  
      let _ = assert (List.rev (prefix_fold_ntree (fun acc x -> x::acc) [] nt) = [1;2;  
      ↪ 3;4;5])
```

```
[34]: val prefix_fold_ntree : ('a -> 'b -> 'a) -> 'a -> 'b ntree -> 'a = <fun>
```

```
[34]: - : unit = ()
```

```
[34]: - : unit = ()
```

```
[34]: - : unit = ()
```

1.2.6 Pre-order fold — version 2 (with fold_left)

This version uses `fold_left` to traverse the list of subtrees and propagates the result with an accumulator parameter. It applies `f acc v` to the root value, stores the result in `acc_v`, and then traverses the list of subtrees with `fold_left` passing `acc_v` as the initial accumulator value. It is not possible to use a map-then-fold pattern because the processing of each subtree depends on previous values.

```
[33]: let rec prefix_fold_ntree f acc nt =  
      match nt with
```

```

| NEmpty -> acc
| NNode (v, l) ->
  let acc_v = f acc v in
  List.fold_left (fun acc nt' -> prefix_fold_ntree f acc nt') acc_v l

let _ = assert (prefix_fold_ntree (+) 0 nt = 15)
let _ = assert (prefix_fold_ntree (fun acc x -> acc@[x]) [] nt = [1; 2; 3; 4;
↪5])
let _ = assert (List.rev (prefix_fold_ntree (fun acc x -> x::acc) [] nt) = [1;2;
↪3;4;5])

```

[33]: val prefix_fold_ntree : ('a -> 'b -> 'a) -> 'a -> 'b ntree -> 'a = <fun>

[33]: - : unit = ()

[33]: - : unit = ()

[33]: - : unit = ()

1.2.7 Pre-order fold — version 3 (point-free)

A more compact version with partial application of the function used in the fold:

```

[32]: let rec prefix_fold_ntree f acc nt =
  match nt with
  | NEmpty -> acc
  | NNode (v, l) -> let acc_v = f acc v in List.fold_left (prefix_fold_ntree f)
↪acc_v l

let _ = assert (prefix_fold_ntree (+) 0 nt = 15)
let _ = assert (prefix_fold_ntree (fun acc x -> acc@[x]) [] nt = [1; 2; 3; 4;
↪5])
let _ = assert (List.rev (prefix_fold_ntree (fun acc x -> x::acc) [] nt) = [1;2;
↪3;4;5])

```

[32]: val prefix_fold_ntree : ('a -> 'b -> 'a) -> 'a -> 'b ntree -> 'a = <fun>

[32]: - : unit = ()

[32]: - : unit = ()

```
[32]: - : unit = ()
```

1.2.8 Building a BST from an n-ary tree

Uses `insert` as the fold function: traverses `nt` in pre-order and inserts each value into an initially empty BST (`Empty`). Demonstrates that `prefix_fold_ntree` is general enough to combine different data types.

```
[42]: type 'a tree = Empty | Node of 'a * 'a tree * 'a tree

let rec insert_bst x t =
  match t with
  | Empty -> Node(x, Empty, Empty)
  | Node(y, l, r) -> if x <= y then Node(y, insert_bst x l, r) else
    ↪Node(y, l, insert_bst x r)

let _ = prefix_fold_ntree (fun bst x -> insert_bst x bst) Empty nt

let flip f = fun x y -> f y x
let _ = prefix_fold_ntree (flip insert_bst) Empty nt
```

```
[42]: type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

```
[42]: val insert_bst : 'a -> 'a tree -> 'a tree = <fun>
```

```
[42]: - : int tree =
Node (1, Empty,
  Node (2, Empty, Node (3, Empty, Node (4, Empty, Node (5, Empty, Empty)))))
```

```
[42]: val flip : ('a -> 'b -> 'c) -> 'b -> 'a -> 'c = <fun>
```

```
[42]: - : int tree =
Node (1, Empty,
  Node (2, Empty, Node (3, Empty, Node (4, Empty, Node (5, Empty, Empty)))))
```